

Banking Operations Observability and Predictive Analytics Platform

ZEBA KHAN

Course Capstone Project Proposal

FullStack Academy AI/ML

Cohort: 2605-FTB-CT-AIM-PT

Instructor: Dr. George Perdrizet

Section 1: Problem Statement

What problem are you trying to solve or what question are you trying to answer?

Imagine working in a bank's operations center. It's Tuesday morning, and suddenly, without warning a critical system goes down. Customers can't access their accounts. Transactions are failing. But here's the frustrating part: by the time anyone realizes there's a problem, it's already too late. The damage is done.

What if you could see the warning signs before the system fails?

Right now, banks have monitoring tools, but they work in silos. Business metrics live in one place, application logs in another, infrastructure data somewhere else entirely. When something goes wrong, it takes hours to piece together what happened and where it started. Teams are scrambling, customers are frustrated, and the bank loses money.

This project tackles that challenge by building a unified system that doesn't just react to problems it predicts them. Imagine a dashboard that sees a problem coming minutes or hours in advance. A system that automatically groups error logs to identify patterns. An AI assistant that explains exactly what went wrong and why.

Why does this problem matter? Who would benefit from a solution?

Banks are under constant pressure. Every minute of downtime costs money and erodes customer trust. If you can predict failures before they happen, you shift from "emergency firefighting" to "smart prevention."

Operations teams would love this. Instead of reacting to crises, they'd be proactive—getting alerts before customers are affected. Business stakeholders care about uptime and customer satisfaction. A system that predicts and prevents outages directly impacts the bottom line.

From a technical standpoint, this project is powerful because it showcases real-world machine learning at scale. It's not just training a model in isolation - building an intelligent system that spans data engineering, predictive analytics, natural language processing, and AI-driven automation. This is the kind of project that gets noticed by fintech companies, banking tech firms, and any organization that deals with operational intelligence.

Section 2: Project Objectives

Primary Goal

Building an intelligent observability platform that transforms banking operations from reactive problem-solving to predictive intelligence - predicting failures, automatically analyzing incidents, and providing actionable insights through a unified dashboard.

Specific Objectives

Predictive Models That See Into the Future Create machine learning models that forecast key banking KPIs - will transaction success rates drop? Is API latency about to spike? Will resource utilization hit critical levels? Using time-series forecasting, classification, and anomaly detection, you'll build models that give operations teams a crystal ball for infrastructure health.

NLP-Powered Log Intelligence Train the system to read application logs like a human would, but at scale. Automatically group similar errors, identify patterns, and classify issues into categories. Instead of teams manually sorting through thousands of log lines, the system learns to do it intelligently.

AI-Driven Root Cause Analysis When something goes wrong, the system shouldn't just report the problem—it should explain it. Build an AI assistant that fetches relevant historical incidents, connects dots across data sources, and generates human-readable explanations of what happened and why.

An Observability Dashboard That Actually Matters Create a web interface that brings everything together. Real-time KPIs, forecasts showing what's coming, AI-generated insights about current issues. Make it something operations teams would actually use, not something that collects dust.

Section 3: Data

What data will you use?

We have three real-world datasets representing the complete picture of banking operations:

transactions_fraud_train.csv captures the business side of millions of transactions with success/failure indicators, processing times, and fraud flags. This shows what customers are experiencing.

infrastructure_metrics.csv is the system's heartbeat - CPU utilization, memory usage, disk I/O, error rates, response times. This reveals what's happening under the hood.

application_logs.csv is the detailed diary of everything that happens - error messages, stack traces, operational events. This is the raw material for understanding *why* problems occur.

Together, these three datasets paint a complete picture of banking operations: business impact, system health, and detailed diagnostics.

Data Source(s)

Primary Source:

<https://drive.google.com/drive/folders/1BCVnbRWzFoITE-88KGH51Zd8kusq-A50>

All three datasets are provided in CSV format, pre-labeled for supervised learning, and ready to use.

Have you verified the data is accessible?

Yes. Data is accessible.

Have you reviewed the data quality and format?

Yes.

Estimated dataset size

Approximately 4MB data size

Data limitations or potential issues

Like all real-world data, these datasets come with challenges. Fraud and critical incidents are rare (1-5% of data), so you'll need techniques like SMOTE or class weighting to prevent models from ignoring the important cases. Infrastructure and transaction data are time-series, meaning standard train/test splits will cause problems—need to use time-based splits. Logs are messy text that requires careful preprocessing. And like all historical data, it reflects past patterns that may not perfectly match the future.

Licensing and usage restrictions

All datasets are provided for educational purposes within this capstone. They won't be shared externally and follow FullStack Academy academic integrity policies.

Section 4: Approach and Methods

Describe your technical approach

I am going to work through the complete machine learning lifecycle, building incrementally:

Understanding the data. Load the three datasets and dive deep. What do the transactions look like? How do infrastructure metrics behave? What patterns hide in the logs? Create visualizations, calculate statistics, and build intuition. Correlate infrastructure metrics with transaction performance: do CPU spikes predict failures? Does memory pressure affect latency?

Building predictive models. Train time-series models to forecast transaction success rates, API latency, and resource utilization. Build classifiers that predict whether an incident will be critical based on KPIs. Implement anomaly detectors using Isolation Forest or other techniques. Evaluate everything rigorously using RMSE, MAE, precision, recall, and F1-score.

Master the logs. Preprocessing and cleaning the messy log text. Extract features using TF-IDF or word embeddings. Apply clustering to group similar errors automatically. Build a knowledge base from labeled historical incidents. By the end, the system should understand what different errors mean.

Building the AI brain. Creating a RAG system that connects your knowledge base with a language model. When something goes wrong, the AI fetches relevant historical incidents and generates explanations. Implement this using LangChain or LangGraph to create an agentic system that can actually reason about incidents.

Putting it all together. Building a Streamlit dashboard that visualizes KPIs, shows forecasts, displays log classifications, and presents AI-generated insights. Create FastAPI endpoints so the models can be called from anywhere. Containerize everything with Docker. Writing documentation. Preparing the presentation.

Tools, libraries, and frameworks

Data & Analysis: Python, Pandas, NumPy, Scikit-learn

Visualizations: Matplotlib, Seaborn

Machine Learning: Scikit-learn for classification and clustering, TensorFlow, PyTorch

NLP: NLTK, SpaCy, word embeddings, Scikit-learn for TF-IDF

AI Integration: LangChain or LangGraph for building the reasoning system, OpenAI API or open-source LLMs (Hugging Face, Ollama)

Dashboard & API: Streamlit for the frontend, FastAPI for the backend

DevOps: Docker and Docker Compose for containerization, Git and GitHub for version control

Development: Jupyter Notebooks for exploration, Python 3.10+, virtual environments

Techniques and algorithms

Time-Series Forecasting: ARIMA for traditional statistical methods, Exponential Smoothing for simpler patterns, Prophet for handling seasonality and trends, LSTM if you want to go deep learning.

Classification: Logistic Regression as baseline, Random Forest for robustness, Gradient Boosting (XGBoost, LightGBM) for maximum predictive power.

Anomaly Detection: Isolation Forest (fast and effective), Local Outlier Factor (density-based), Z-score and IQR methods (simple and interpretable).

NLP & Clustering: Tokenization and text preprocessing to clean logs, TF-IDF for traditional feature extraction, Word embeddings for semantic understanding, K-Means for simple clustering.

AI Integration: Retrieval-Augmented Generation to fetch relevant knowledge, prompt engineering to guide the LLM, agentic workflows that let the AI take actions based on reasoning.

Section 5: Expected Deliverables

What will you produce by the end of this project?

Jupyter Notebooks documenting your entire journey—01_data_exploration.ipynb showing what you discovered, 02_predictive_modeling.ipynb with all your models, 03_nlp_log_analysis.ipynb documenting the NLP pipeline. These aren't just code—they're a narrative of your process.

Trained Models saved and ready to use— forecasting model, classification model, anomaly detector. These live in a models/ folder and can be loaded instantly for predictions.

Working Backend API built with FastAPI. Endpoints for making predictions, classifying logs, and triggering AI analysis. This is production-grade thinking—the API could theoretically serve a real banking system.

Interactive Dashboard built with Streamlit. Real-time KPI displays, forecast visualizations, log classification results, AI-generated incident summaries. Something that operations teams would actually open and use.

Docker Setup with Dockerfile and docker-compose.yml. The entire system containerized, reproducible, deployable. Push a button and it runs.

Comprehensive Documentation. README with setup instructions, technical documentation explaining architecture and decisions, results summary with findings and limitations, deployment guide for running the system.

GitHub Repository professionally organized with clear folder structure, meaningful commit messages, proper branching strategy. Something you'd be proud to show an employer.

Final Presentation to the cohort—10-15 minutes demonstrating your system, showing results, explaining your approach, discussing what you learned.

Section 6: Success Criteria

How will you know if your project is successful?

The predictive models will work. Time-series forecasting achieves RMSE less than 10% of mean values. Classification models hit F1-score of at least 0.70. Anomaly detection catches 80% of known anomalies with false positive rate below 10%.

Log analysis will work. The clustering produces 3+ distinct, coherent clusters. Silhouette score exceeds 0.5. When manually checked, at least 80% of logs are correctly grouped by issue type.

Dashboard will work. All visualizations render correctly. Data updates without errors. Predictions appear instantly. AI-generated insights appear within 5 seconds.

API will work. All endpoints return valid responses. Model inference completes in under 2 seconds. The deployment will work. Docker image builds without errors. The system runs locally without manual intervention.

Section 7: Known Risks and Challenges

What obstacles do you anticipate? How will you address them?

Imbalanced Data (Fraud and Critical Incidents are Rare). Many datasets in banking have fraud or critical incidents occurring in only 1-5% of cases. Models might learn to just predict the majority class and miss the important cases entirely.

Temporal Data and Train/Test Contamination. Infrastructure metrics and transactions are time-series. If you split randomly, the model sees future data during training, which is cheating and makes it perform worse in reality.

Messy Logs Require Careful NLP Processing. Application logs are unstructured, inconsistently formatted, contain duplicates and irrelevant information. Processing them takes iteration and refinement.

LLM/RAG Integration Can Get Complex. Connecting external LLMs for root cause analysis might introduce latency, cost, or architectural complexity.

Deployment Has Lots of Moving Parts. Docker setup, environment configuration, dependencies—lots can go wrong.

Data Quality Issues in Real-World Data. Missing values, outliers, mislabeled data—it's all normal.

The project has many components—modeling, NLP, RAG, dashboard, deployment.

Computing Power. LSTM models or embedding computation on a CPU-only machine can be slow.

Section 8: Resources Needed

What resources, access, or support will you need?

Data & Access: the CSV files via Google Drive. GitHub account for version control

Development Environment: Jupyter environment—either local installation or Google Colab. Python 3.10+ with virtual environments

Tools & Libraries: Python libraries (Pandas, Scikit-learn, Streamlit, FastAPI, PyTorch, etc.).

API Access: OpenAI API or open-source LLMs from Hugging Face

Deployment: Docker

Section 9: GitHub Repository

Repository URL:

<https://github.com/zeba49608-beep/BankingOperationsObservabilityandPredictiveAnalyticsPlatform?tab=readme-ov-file>

Advisor Feedback

(To be completed by instructor and/or TA)

Date Reviewed: _____

Approval Status:

- ☐ Approved
- ☐ Approved with minor revisions
- ☐ Needs revision

- **Comments:**
